

DDL na prática

O que é DDL?

DDL significa **Data Definition Language**, ou **Linguagem de Definição de Dados**.

Ela é utilizada para definir e modificar a estrutura do banco de dados, como:

- Tabelas;
- Colunas;
- Tipos de dados;
- Restrições;
- Índices;
- Visões;
- Esquemas.

Os comandos DDL normalmente afetam a **estrutura** do banco, e não diretamente os registros armazenados.

Principais comandos DDL

- **CREATE**

- Usado para criar objetos no banco de dados.

Criando uma tabela

```
CREATE TABLE produtos (  
    id INT,  
    nome VARCHAR(100),  
    preco DECIMAL(10, 2)  
);
```

Criando uma tabela com chave primária

```
CREATE TABLE produtos (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100) NULL,  
    preco DECIMAL(10, 2) NOT NULL  
);
```

- **ALTER**

Usado para modificar a estrutura de um objeto existente.

Pode ser utilizado para:

- Adicionar colunas;
- Remover colunas;
- Alterar tipos de dados;

- Adicionar restrições;
- Remover restrições;
- Renomear tabelas ou colunas, dependendo do SGBD.

Adicionando uma coluna

```
ALTER TABLE produtos ADD estoque INT;
```

Removendo uma coluna

```
ALTER TABLE produtos DROP COLUMN estoque;
```

Adicionando uma restrição

```
ALTER TABLE produtos ADD CONSTRAINT pk_produtos PRIMARY KEY (id);
```

```
ALTER TABLE produtos MODIFY nome VARCHAR(100) NOT NULL;
```

*-- Antes de adicionar **NOT NULL**, a tabela não pode conter valores nulos nessa coluna.*

```
ALTER TABLE produtos ADD CONSTRAINT uq_produtos_nome UNIQUE (nome);
```

- **DROP**

Usado para excluir permanentemente um objeto do banco de dados.

```
DROP TABLE produtos;
```

Esse comando remove:

- A tabela;
- As colunas;
- Os registros;
- As restrições;
- Os índices associados, conforme o SGBD.
- **O comando DROP TABLE é destrutivo. Deve ser usado com bastante cuidado.**

Criando a tabela

```
CREATE TABLE produtos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10, 2) NOT NULL,  
    codigo_barra VARCHAR(80) UNIQUE  
);
```

Restrições utilizadas

- PRIMARY KEY: identifica cada produto de forma única;
- AUTO_INCREMENT: gera automaticamente o valor de id;
- NOT NULL: impede valores nulos;
- UNIQUE: impede códigos de barras repetidos.

Removendo uma coluna

Para remover a coluna codigo_barra:

```
ALTER TABLE produtos DROP COLUMN codigo_barra;
```

Explicação

O comando DROP COLUMN remove a coluna da tabela.

Nesse caso:

- A coluna codigo_barra será excluída;
- A restrição UNIQUE associada também será removida;
- Todos os códigos de barras armazenados serão perdidos;
- As demais colunas permanecerão na tabela.

Removendo a restrição UNIQUE

Para demonstrar a remoção da restrição UNIQUE, a tabela precisa possuir novamente a coluna codigo_barra.

```
ALTER TABLE produtos ADD COLUMN codigo_barra VARCHAR(80) UNIQUE;
```

Agora, podemos remover apenas a restrição UNIQUE, mantendo a coluna:

```
ALTER TABLE produtos DROP INDEX codigo_barra;
```

Explicação

No MySQL, uma restrição UNIQUE é implementada por meio de um índice exclusivo.

Por isso, usamos:

```
DROP INDEX codigo_barra;
```

Nesse caso:

- A coluna codigo_barra continua existindo;
- A regra que impedia valores repetidos é removida;
- Passa a ser possível cadastrar códigos de barras iguais.

Por exemplo, depois de remover a restrição:

```
INSERT INTO produtos (nome, preco, codigo_barra) VALUES ('Teclado', 100.00, 'ABC123');
INSERT INTO produtos (nome, preco, codigo_barra) VALUES ('Mouse', 50.00, 'ABC123');
```

Os dois registros poderão ser inseridos, pois o valor ABC123 deixou de ser exclusivo.

Removendo a chave primária

Na tabela criada, a chave primária está definida na coluna id:

```
id INT PRIMARY KEY AUTO_INCREMENT
```

Para removê-la:

```
ALTER TABLE produtos DROP PRIMARY KEY;
```

Explicação

Esse comando remove a chave primária da tabela.

Depois da execução:

- id deixará de ser uma chave primária;
- A tabela não terá mais uma identificação única obrigatória;
- Poderão existir valores repetidos em id;
- O AUTO_INCREMENT deixará de funcionar corretamente sem uma chave ou índice adequado, dependendo da configuração do MySQL.

A coluna id continuará existindo, mas perderá a restrição de chave primária.

Removendo NOT NULL

A tabela possui as seguintes colunas obrigatórias:

```
nome VARCHAR(100) NOT NULLpreco DECIMAL(10, 2) NOT NULL
```

Para permitir valores nulos em nome, devemos redefinir a coluna:

```
ALTER TABLE produtos MODIFY nome VARCHAR(100) NULL;
```

Para permitir valores nulos em preco:

```
ALTER TABLE produtos MODIFY preco DECIMAL(10, 2) NULL;
```

Explicação

No MySQL, NOT NULL não é removido com:

DROP CONSTRAINT

A coluna deve ser modificada com MODIFY.

Depois disso, estes comandos poderão ser aceitos:

```
INSERT INTO produtos (nome, preco) VALUES (NULL, 100.00);  
INSERT INTO produtos (nome, preco) VALUES ('Teclado', NULL);
```

Antes da alteração, ambos seriam rejeitados porque nome e preco eram obrigatórios.

TRUNCATE

Usado para remover todos os registros de uma tabela, mantendo sua estrutura.

```
TRUNCATE TABLE produtos;
```

Diferença entre DROP e TRUNCATE

Comando	Remove dados	Remove estrutura
DROP TABLE	Sim	Sim
TRUNCATE TABLE	Sim	Não
DELETE FROM	Sim	Não

Exemplo

Depois de executar:

```
TRUNCATE TABLE produtos;
```

A tabela ainda existirá:

```
SELECT * FROM produtos;
```

Porém, não haverá registros.

- RENAME

O comando:

```
ALTER TABLE produtos RENAME TO itens;
```

renomeia a tabela produtos para itens.

Antes da execução

A tabela possui o nome:

produtos

Por exemplo:

```
SELECT * FROM produtos;
```

Executando a alteração

```
ALTER TABLE produtos RENAME TO itens;
```

Depois da execução

A tabela passa a se chamar:

itens

A consulta correta será:

```
SELECT * FROM itens;
```

A consulta abaixo causará erro:

```
SELECT * FROM produtos;
```

Isso acontece porque a tabela produtos não existe mais com esse nome.

O que é preservado?

Ao renomear a tabela, o MySQL mantém:

- As colunas;
- Os registros;
- A chave primária;
- O AUTO_INCREMENT;
- As restrições NOT NULL;
- As restrições UNIQUE;
- Os índices;
- As chaves estrangeiras, respeitando as regras do banco.

O que muda é apenas o nome da tabela.

Verificando o resultado

Para listar as tabelas existentes:

```
SHOW TABLES;
```

Para consultar a estrutura da tabela renomeada:

```
DESCRIBE itens;
```

Também podemos visualizar sua definição completa:

```
SHOW CREATE TABLE itens;
```

Observação

No MySQL, também é possível usar:

```
RENAME TABLE produtos TO itens;
```

As duas formas renomeiam a tabela:

```
ALTER TABLE produtos RENAME TO itens;  
RENAME TABLE produtos TO itens;
```

Para o exemplo apresentado, ambas são válidas no MySQL.